# Text Classification Experiment Report

## Comparative Analysis of Text Representation Methods for News Classification

---

## 1. Introduction

### 1.1 Background

Text classification is a fundamental task in Natural Language Processing (NLP) that involves assigning predefined categories to text documents. The performance of text classification models heavily depends on how text is represented numerically. This experiment explores three major categories of text representation methods:

1. **Traditional Bag-of-Words (BoW)** approaches
2. **Word Embedding methods** (Word2Vec, GloVe)
3. **Pre-trained Language Models** (BERT)

### 1.2 Objective

This experiment aims to: - Understand the fundamental principles of different text representation methods - Compare the performance of various approaches on news text classification - Analyze the strengths and weaknesses of each method

---

## 2. Experimental Setup

### 2.1 Dataset

**New York Times (NYT) News Dataset** - Total samples: 11,519 - Number of classes: 3 (sports, politics, business) - Class distribution: - Sports: 8,639 (75.0%) - Politics: 1,451 (12.6%) - Business: 1,429 (12.4%)

**AG News Dataset** (for Word2Vec training) - Total samples: 90,000 - Used for training domain-specific Word2Vec embeddings

### 2.2 Data Split

To ensure reproducibility and fair comparison across all experiments: - Training set: 80% (9,215 samples) - Test set: 20% (2,304 samples) - Random state: 42 - Stratified split to maintain class distribution

### 2.3 Evaluation Metrics

- **Accuracy**: The proportion of correctly classified samples out of all test samples
- **Macro-F1 Score**: The average of F1 scores across all classes, treating each class equally regardless of its size

---

## 3. Methods

### 3.1 Task 1: Bag of Words (30 points)

Bag-of-Words is a traditional text representation method that treats each document as a "bag" of words, ignoring grammar and word order but keeping multiplicity.

**3.1.1 Binary Bag of Words** Each document is represented as a binary vector where: - `x_i = 1` if the i-th word in the vocabulary appears in the document - `x_i = 0` otherwise

**3.1.2 Word Frequency (Term Frequency)** Instead of binary values, we count the actual occurrences of each word: - `x_i = c_i` where `c_i` is the count of word i in the document

**3.1.3 Classifier** All BoW methods use **Logistic Regression** as the classifier with max iterations = 1000.

### 3.2 Task 2: Word Embeddings (50 points)

Word embeddings represent words as dense vectors in a continuous space, capturing semantic relationships.

#### 3.2.1 Pre-trained GloVe Embeddings

- Source: GloVe.6B (100-dimensional)
- Vocabulary size: 400,000 words
- Document representation: Average of all word vectors in the document
- Formula: `e(d) = (1/n) × Σ  e(w )`

#### 3.2.2 Word2Vec Trained on AG News

- Framework: Gensim Word2Vec
- Vector size: 100 dimensions
- Window size: 5
- Min count: 1
- Epochs: 10
- Trained on 90,000 AG News documents

#### 3.2.3 Word2Vec Trained on NYT

- Same hyperparameters as 3.2.2
- Trained on 9,215 NYT training documents

**3.2.4 Classifier** All word embedding methods use **Logistic Regression** as the classifier.

### 3.3 Task 3: Pre-trained Neural Models (20 points)

#### 3.3.1 BERT Fine-tuning

- Model: `google-bert/bert-base-uncased`
- Max sequence length: 64
- Number of epochs: 3
- Batch size: 16
- Optimizer: AdamW with weight decay 0.01
- Learning rate: Linear warmup with 500 warmup steps

---

## 4. Results

### 4.1 Task 1: Bag of Words Results

| Method | Accuracy | Macro-F1 |
|---|---|---|
| Binary Bag of Words | 0.9848 | 0.9634 |
| Word Frequency | **0.9870** | **0.9687** |
| Count Vector | 0.9861 | 0.9677 |

**Analysis**: Word Frequency achieved the best performance among BoW methods. The rich vocabulary (61,021 words) combined with term frequency information provides strong discriminative power for this news classification task.

### 4.2 Task 2: Word Embeddings Results

| Method | Accuracy | Macro-F1 |
|---|---|---|
| GloVe (Pre-trained) | **0.9813** | **0.9573** |
| Word2Vec (AG News) | 0.9774 | 0.9492 |
| Word2Vec (NYT) | 0.9783 | 0.9484 |

**Analysis**: Pre-trained GloVe embeddings performed best among word embedding methods. The 400K vocabulary and contribute to its superior performance. Interestingly, Word2Vec trained on NYT (in-domain) did not significantly outperform Word2Vec trained on AG News (out-of-domain).

### 4.3 Task 3: BERT Results

| Method | Accuracy | Macro-F1 |
|---|---|---|
| BERT (Fine-tuned) | 0.9766 | 0.9469 |

**Analysis**: BERT achieved competitive results but did not outperform simpler BoW methods. Possible reasons: - Max sequence length of 64 may truncate important information in longer news articles - The pre-training task (Masked Language Modeling) differs from the classification task - Relatively small dataset (9,215 training samples) may limit fine-tuning effectiveness

### 4.4 Overall Comparison

| Category | Method | Accuracy | Macro-F1 |
|---|---|---|---|
| **Task 1** | Word Frequency | **0.9870** | **0.9687** |
| **Task 1** | Count Vector | 0.9861 | 0.9677 |
| **Task 1** | Binary BoW | 0.9848 | 0.9634 |
| **Task 2** | GloVe (Pre-trained) | 0.9813 | 0.9573 |
| **Task 2** | Word2Vec (NYT) | 0.9783 | 0.9484 |
| **Task 2** | Word2Vec (AG News) | 0.9774 | 0.9492 |
| **Task 3** | BERT (Fine-tuned) | 0.9766 | 0.9469 |

---

## 5. Analysis and Discussion

### 5.1 Why BoW Outperformed Other Methods?

1. **Dataset Characteristics**: The NYT dataset has a highly imbalanced class distribution (75% sports). BoW methods with Logistic Regression can effectively learn class-specific keywords (e.g., "score", "game" for sports; "government", "election" for politics).
2. **Feature Density**: BoW creates sparse but highly informative feature vectors (61,021 dimensions) that directly capture topic-specific vocabulary.
3. **Simplicity**: For tasks where keyword presence is highly predictive, simple methods often work as well as or better than complex ones.

### 5.2 Word Embedding Limitations

1. **Information Loss**: Averaging word vectors loses word order and important context
2. **Dimension Constraint**: 100-dimensional vectors may not capture all semantic nuances

3. **Domain Mismatch**: Pre-trained embeddings may not perfectly fit news domain vocabulary

### 5.3 BERT Considerations

1. **Contextual Embeddings**: BERT generates context-dependent representations, but averaging [CLS] token or all tokens loses this advantage
2. **Short Sequence Length**: max_length=64 may truncate important information
3. **Training Data Size**: Fine-tuning on ~9K samples may lead to underfitting or overfitting

---

## 6. Conclusion

This experiment compared three categories of text representation methods for news classification:

1. **Bag-of-Words methods** achieved the best overall performance (Accuracy: 98.70%, Macro-F1: 96.87%)
2. **Word embeddings** showed solid performance but did not surpass BoW (Accuracy: 97.74-98.13%)
3. **BERT fine-tuning** achieved competitive results (Accuracy: 97.66%) but faced limitations with short sequence length and small training data

**Key Findings**: - For text classification tasks with clear category-specific vocabulary and imbalanced classes, traditional BoW methods can outperform more complex neural approaches - Pre-trained embeddings provide good generalization but may not always outperform task-specific features - Modern pre-trained models like BERT require sufficient training data and appropriate hyperparameter settings to achieve optimal performance

---

## 7. Code Structure

```
HW-1/
  homework1.py              # Task 1: Bag of Words
  homework1_word2vec.py     # Task 2.2 & 2.3: Word2Vec Training
  homework1_glove.py        # Task 2.1: GloVe
  homework1_bert.py         # Task 3: BERT Fine-tuning
  nyt.csv                   # NYT Dataset
  ag.csv                    # AG News Dataset
  task1_results.csv         # Task 1 Results
  task2_results.csv         # Task 2 Word2Vec Results
  task2_glove_results.csv   # Task 2 GloVe Results
  task3_results.csv         # Task 3 Results
```

## 8. Running Instructions

**Task 1: Bag of Words**

```
python homework1.py
```

**Task 2.1: GloVe (requires glove.6B.100d.txt)**

```
python homework1_glove.py
```

**Task 2.2 & 2.3: Word2Vec**

```
python homework1_word2vec.py
```

**Task 3: BERT Fine-tuning (requires bert-base-uncased model)**

```
python homework1_bert.py
```

**Note**: For Task 3, download the BERT model from https://huggingface.co/google-bert/bert-base-uncased to a local directory and update `BERT_MODEL_PATH` in the script.